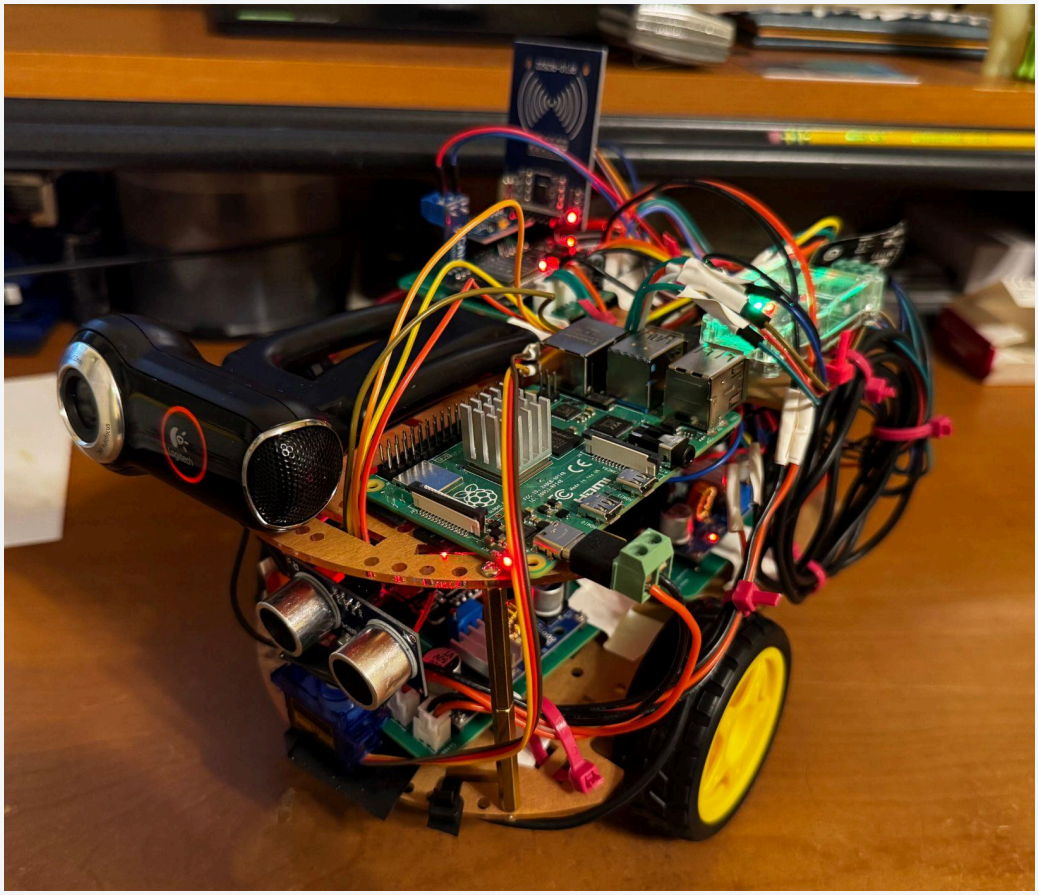
Distributed Electric Vehicle Control and Autonomy Platform

Design, Integration, and Validation of a CAN-Based Embedded Vehicle System

Project Period: July 11 - August 17, 2026
Documentation completed August 18, 2026
Prepared by Alexander Livson



Final small-scale electric vehicle development platform integrating three custom ECU PCBs, a Raspberry Pi supervisory controller, distributed CAN communication, sensing, safety logic, and vision-assisted autonomy.

Portfolio / Engineering Design Report

Contents

1. Executive Summary
2. Project Motivation and Goals
3. System Architecture
4. Hardware and Electrical Design
5. ECU Design
6. CAN Communication Architecture
7. Custom PCB Development
8. Raspberry Pi Supervisory Software
9. Drive Modes and Autonomous Behavior
10. Safety Architecture
11. Web Dashboard and Human-Machine Interface
12. Integration, Debugging, and Engineering Challenges
13. Testing and Validation
14. Development Timeline
15. Engineering Lessons Learned
16. Current Limitations and Future Revision
17. Conclusion

Appendix A. Final System Specifications
Appendix B. CAN Message Reference
Appendix C. Development Tools and Authorship

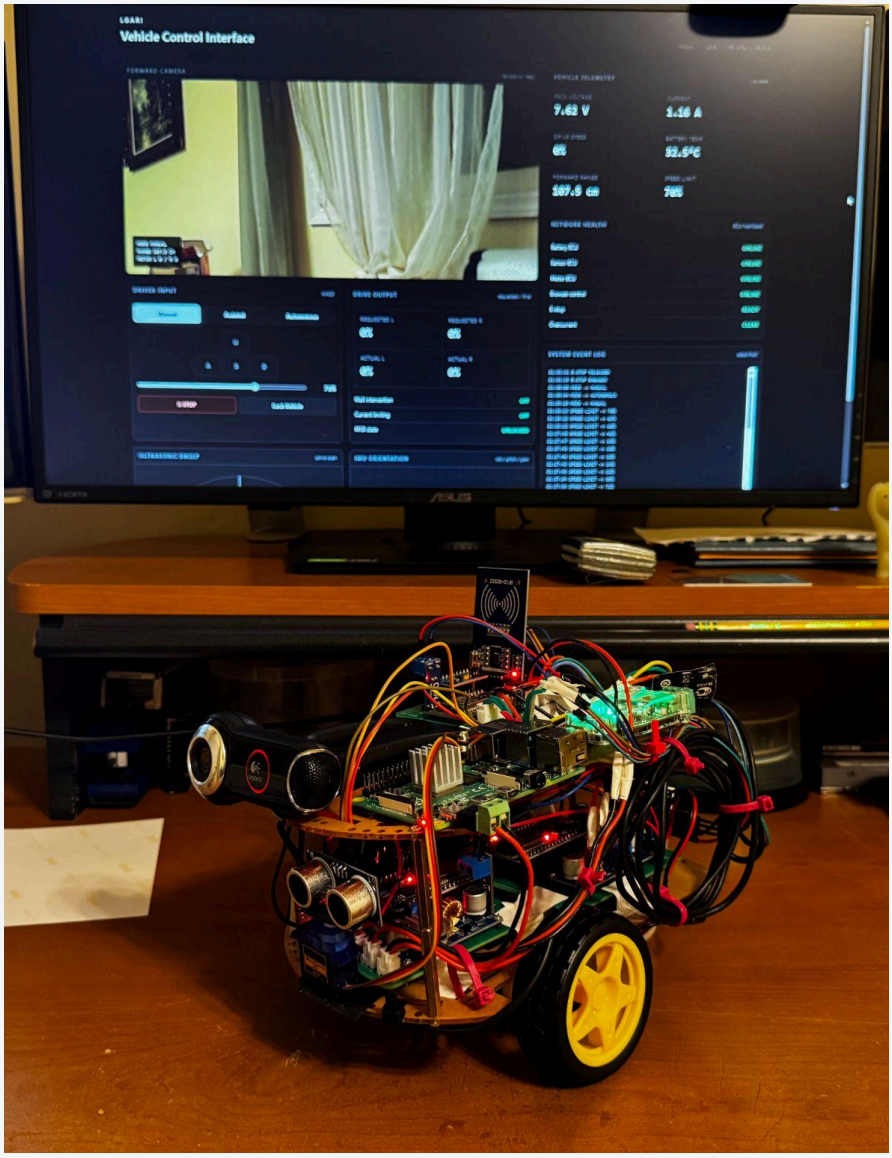
1. Executive Summary

This project is a smaller-scale electric vehicle development platform built to study the same kinds of embedded systems, communication methods, safety logic, power monitoring, and supervisory control concepts used in larger electric vehicles. I started the project on July 11, 2026 with a relatively simple goal: build a small vehicle with multiple electronic control units that could communicate over CAN. Over the next five weeks, the project grew into a four-node control system with three custom ESP32-based ECU PCBs, a Raspberry Pi vehicle computer, a 500 kbps CAN network, a browser-based driver interface, RFID authorization, battery monitoring, ultrasonic collision protection, and OpenCV-assisted autonomous obstacle avoidance.

The project was primarily a learning platform. My biggest technical goal was to become comfortable designing PCBs from scratch, but I also wanted hands-on experience with embedded firmware, Raspberry Pi development, CAN communication, networking, computer vision, power distribution, and system-level integration. Before this project I had not worked with CAN, had not built a distributed multi-ECU system, and had very limited PCB design experience. By the end of the project, I had designed, manufactured, soldered, debugged, and integrated three custom two-layer PCBs into a working vehicle.

The final architecture imitates a simplified electric vehicle. A Battery ECU monitors pack voltage, current, and temperature. A Sensor ECU handles the IMU, rotating ultrasonic sensor, RFID reader, OLED displays, and LED matrix. A Motor ECU controls two drive motors. A Raspberry Pi 4 Model B acts as the supervisory vehicle computer, receiving telemetry over CAN, enforcing high-level safety rules, hosting the control dashboard, and running autonomous perception. If critical information becomes stale, the vehicle stops rather than continuing on outdated data.

The completed prototype measures approximately 24 cm by 19 cm by 19 cm and weighs about 872 g including its battery. It uses a 2S 7.4 V, 2000 mAh LiPo pack, draws roughly 0.8 to 0.9 A at idle, 1.2 to 1.3 A during typical driving, and reached an observed peak of about 1.6 A in normal testing. Practical runtime is approximately one hour. The vehicle is comfortable to operate at full command, although most testing was performed around a 70% speed limit. Maximum physical speed is estimated at roughly 4 ft/s, but because the system does not use wheel encoders, the dashboard intentionally reports motor-output percentage rather than claiming measured vehicle velocity.



Completed vehicle and web-based supervisory dashboard operating together during final integration.

Key Outcomes

Area	Final Outcome
Distributed control	Three ESP32 ECUs plus Raspberry Pi supervisory controller
Vehicle network	Four-node 500 kbps CAN bus
PCB development	Three custom 2-layer through-hole-oriented ECU PCBs designed in KiCad and manufactured by JLCPCB
Driver control	Low-latency browser-based WASD control with a speed limit slider and E-stop
Safety	RFID authorization, wall intervention, stale-telemetry stop, overcurrent limiting/shutdown, browser watchdog
Autonomy	Ultrasonic proximity sensing combined with OpenCV left/right visual obstruction analysis
Final control latency	Approximately 20-30 ms over direct IPv4 access on the local network
Project cost	Approximately $300

2. Project Motivation and Goals

2.1 Why I Built It

I built this project because I wanted to understand electric vehicle electronics at a much deeper level than I could get from reading about them. I am especially interested in vehicle electronics, embedded firmware, battery systems, and the way distributed controllers work together inside a larger system. I also wanted the project to give me practical background that would transfer into engineering clubs and future vehicle projects.

The initial concept was much smaller. I wanted one to three ECUs to communicate with each other in a car-like system. The Raspberry Pi, web dashboard, camera, computer vision, and rotating ultrasonic sensor were added later as the project evolved. Once the basic ECU architecture worked, I wanted a higher-level computer that could combine the data from the embedded controllers and make decisions in a way that felt closer to a real vehicle.

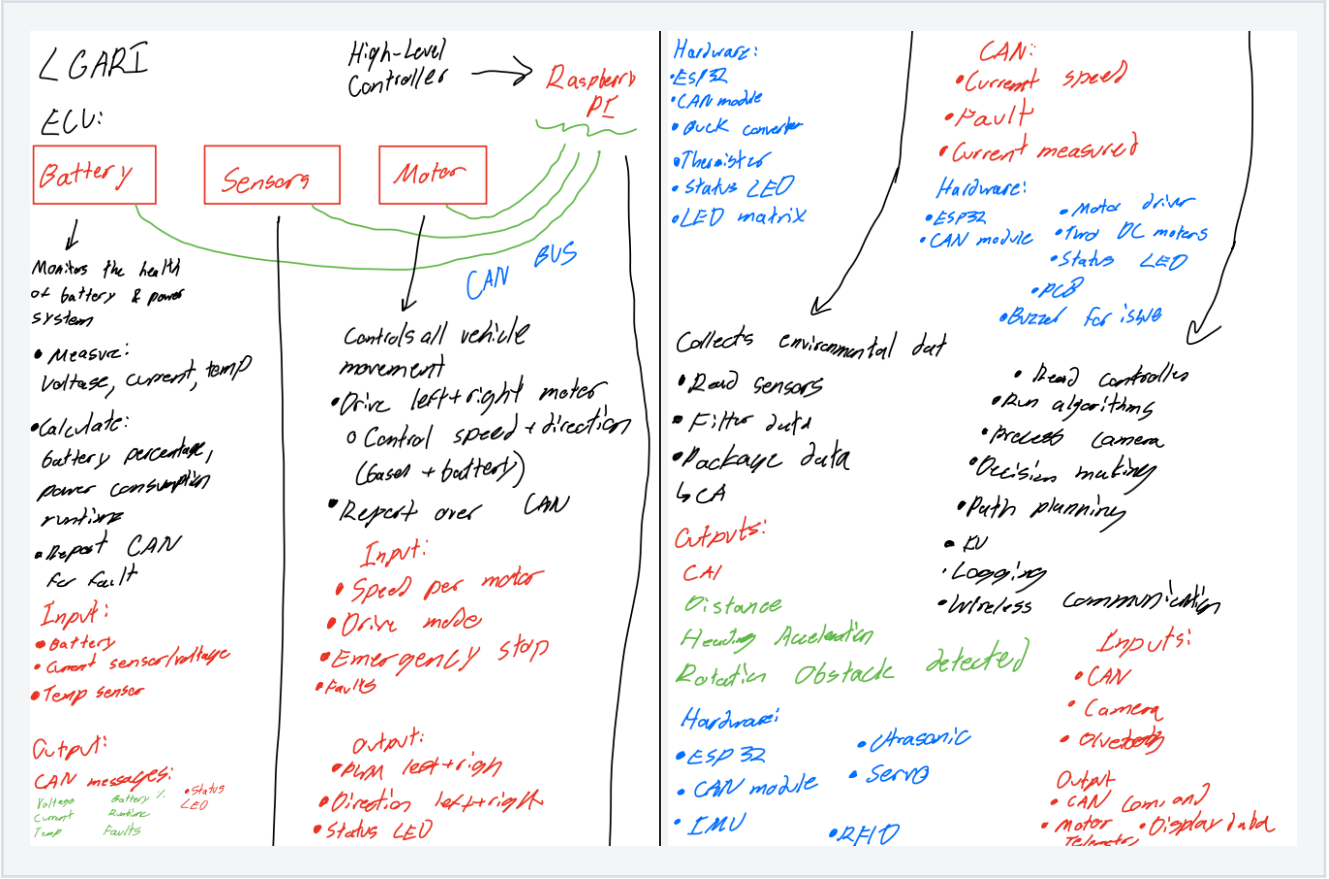
2.2 Primary Learning Objectives

- **PCB design:** Learn the full process from schematic capture and custom footprints through routing, DRC, fabrication, hand soldering, bring-up, and revision planning.
- **CAN communication:** Build and debug a real multi-node wired vehicle network using an industry-standard automotive protocol.
- **Embedded systems:** Write firmware for separate controllers with clearly divided responsibilities and shared message definitions.
- **Raspberry Pi development:** Use Linux, SocketCAN, systemd, Python, networking, camera capture, and a local web server as a vehicle computer.
- **Safety-oriented control:** Treat safety logic as a separate supervisory layer that can override user or autonomous requests.
- **Computer vision and autonomy:** Add perception that complements direct distance sensing.

2.3 Design Philosophy

Three main ideas guided the project: learning, safety, and realistic vehicle architecture. I did not want to build a normal RC car with a single microcontroller and then attach unrelated features to it. I wanted the electronics to be organized as a distributed system in which each controller had a clear job, each important subsystem reported its state, and the vehicle computer made decisions using information from the entire network.

That philosophy also influenced fault behavior. If a critical ECU stops reporting, the preferred response is to pause motion rather than assume the last value is still valid. If the vehicle is locked, the motors stay disabled. If the current exceeds the configured limits, the supervisory controller reduces or removes drive commands. Even Manual mode retains a forward collision stop because the system should still protect itself during debugging.



Early system architecture concept showing separate Battery, Sensor, and Motor ECUs connected to a Raspberry Pi over CAN.

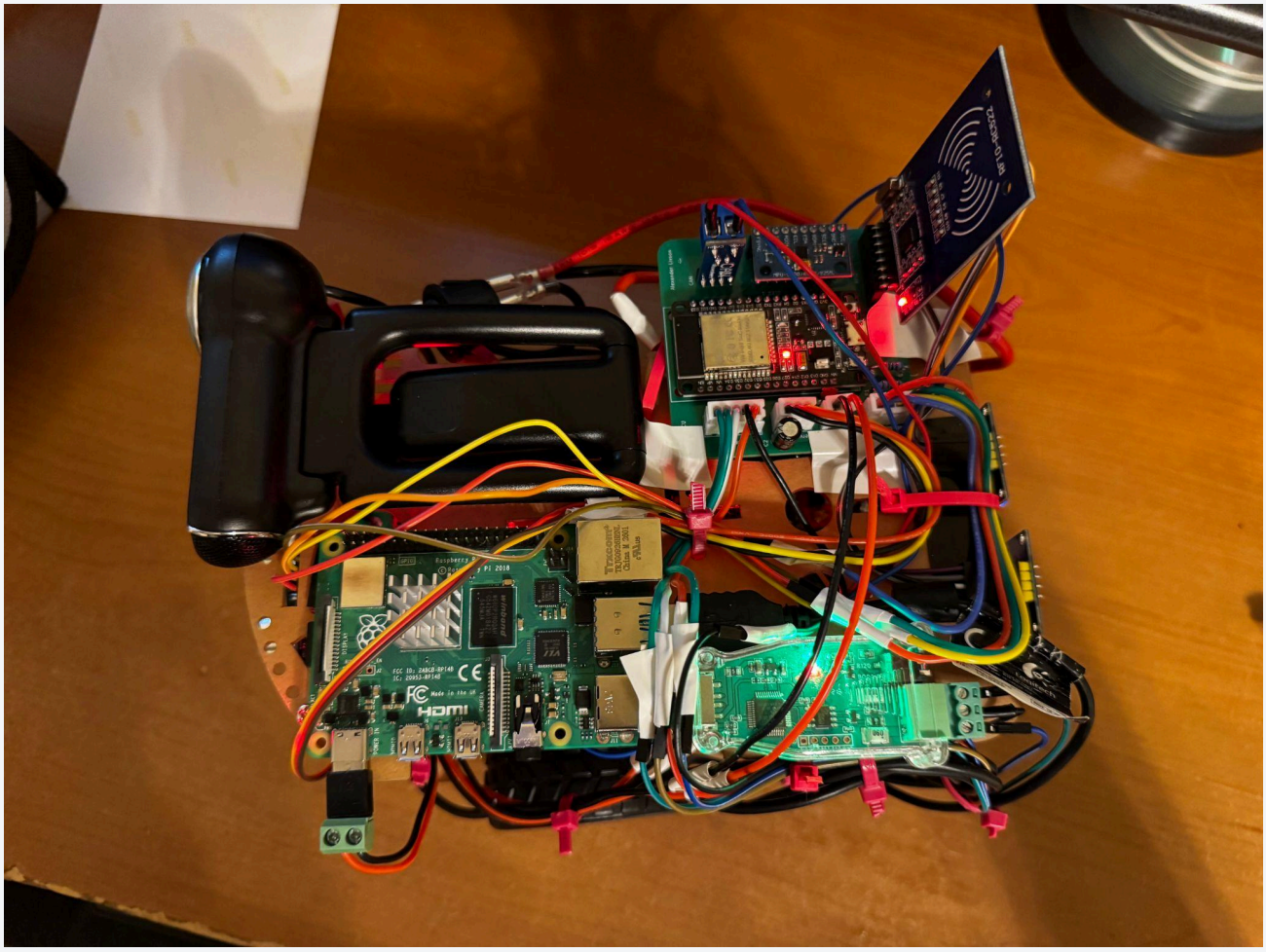
3. System Architecture

3.1 Distributed Vehicle Architecture

The final system has four communicating nodes: a Battery ECU, Sensor ECU, Motor ECU, and Raspberry Pi 4 Model B. The ESP32 ECUs handle deterministic local hardware tasks, while the Raspberry Pi acts as a vehicle computer and supervisory controller. The Pi hosts the driver interface, receives telemetry, applies safety rules, commands the Motor ECU, and runs OpenCV perception.

I chose multiple ESP32 controllers instead of one large controller for both practical and architectural reasons. The Sensor ECU alone uses a large number of GPIO pins, and a single ESP32 would not have had enough convenient pins for all peripherals. Separating the system also made the design easier to organize and debug. Battery monitoring, sensing, and motor actuation could each be validated independently before the complete vehicle was assembled.

The Raspberry Pi was added because the project had grown beyond what I wanted an ESP32 to manage. The Pi is much better suited for USB cameras, OpenCV, a web server, Linux networking, and higher-level decision making. It also provides a natural place to combine information from multiple ECUs without moving every sensor and actuator onto one controller.



High-level architecture: Raspberry Pi supervisory controller connected by CAN to Battery ECU, Sensor ECU, and Motor ECU.

3.2 Functional Responsibilities

Controller	Primary Responsibilities
Battery ECU	Whole-vehicle pack voltage, current, and temperature monitoring; battery health/status reporting over CAN; local status indication.
Sensor ECU	Ultrasonic ranging and servo scan, IMU telemetry, RFID lock state, OLED vehicle displays, LED matrix direction indication, sensor status reporting.
Motor ECU	Dual TT motor actuation through TB6612FNG driver, left/right output control, status feedback, LED/buzzer fault indication.
Raspberry Pi 4	Vehicle computer, SocketCAN interface, driver web server, safety manager, drive-mode logic, telemetry aggregation, OpenCV perception, autonomous decision making.

3.3 Why CAN

I wanted reliable wired communication rather than making the core vehicle controls depend on Wi-Fi. CAN is a two-wire differential bus, is widely used in automotive systems, and is well suited to multiple controllers sharing one network. The ESP32 also provides useful hardware support for CAN-compatible communication when paired with an external transceiver. Using CAN gave the project an architecture that is much closer to real vehicle electronics than a group of point-to-point UART links.

4. Hardware and Electrical Design

4.1 Final Hardware

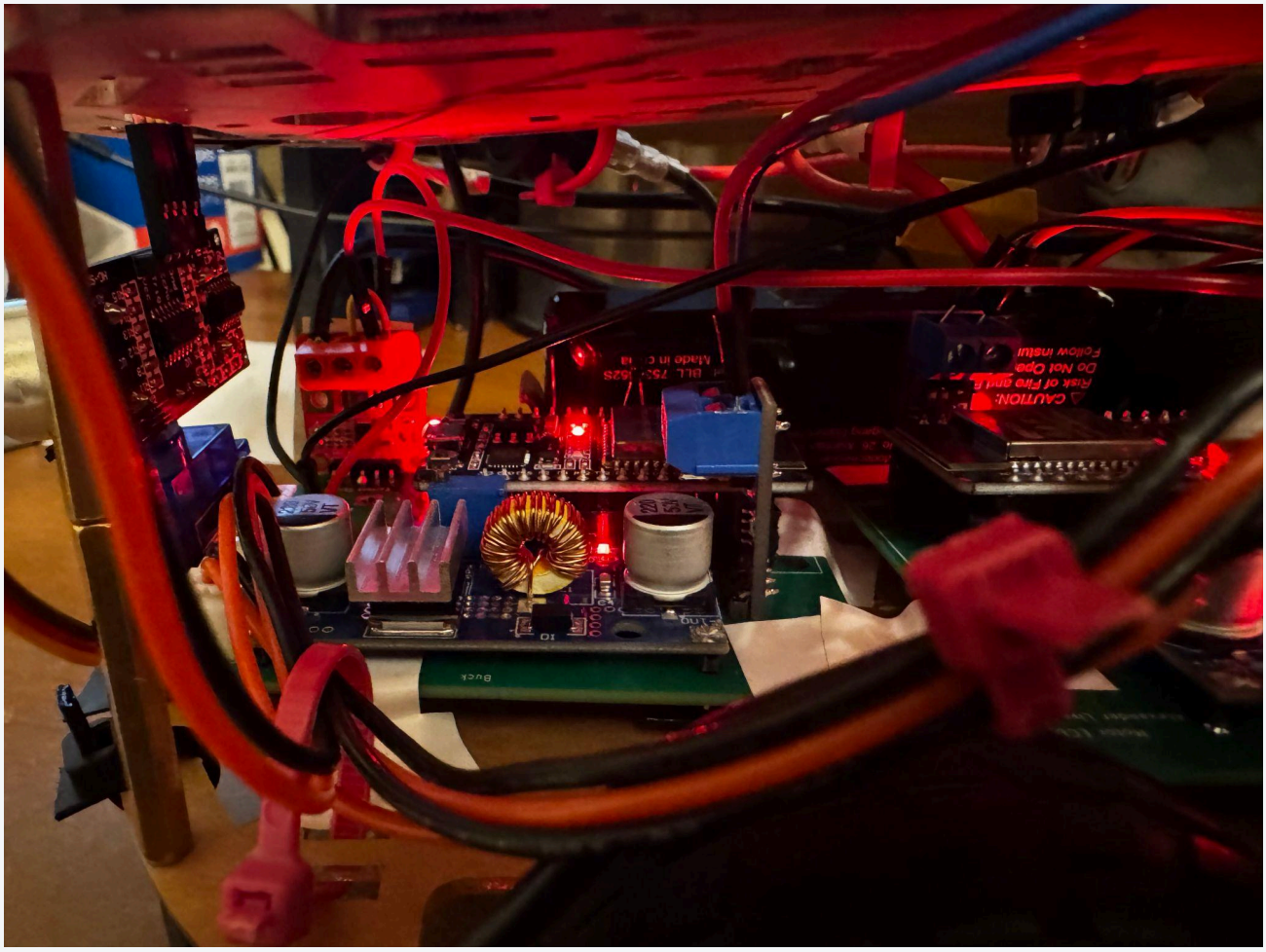
Subsystem	Hardware
Vehicle computer	Raspberry Pi 4 Model B, 1 GB RAM, Raspberry Pi OS Lite
ECU controllers	3x ESP32-WROOM-32 development boards
CAN	3.3 V CAN transceiver modules on ESP32 ECUs; Raspberry Pi CAN interface; 500 kbps
Battery	2S LiPo, 7.4 V nominal, 2000 mAh, 25C, approximately 14.8 Wh
Current monitoring	INA228 current/voltage monitor
Temperature	10 kOhm NTC thermistor
Drive	2x TT gearmotors, TB6612FNG dual motor driver
Distance sensing	HC-SR04 ultrasonic sensor on hobby microservo
IMU	MPU9250 module used primarily as an MPU6500-class accel/gyro device
RFID	MFRC522 reader and compatible RFID card/tag
Displays	2x 0.96 in 128x64 OLED displays; MAX7219 8x8 LED matrix
Camera	Forward-facing legacy Logitech USB webcam
Indicators	Red status LEDs and buzzer
Power conversion	2x approximately 5 A buck converters, both configured for 5 V output

4.2 Power Architecture

The final vehicle uses a 2S LiPo battery with a nominal voltage of 7.4 V. The pack feeds a fuse and main power switch before entering the Battery ECU. The INA228 is placed at the front of the electrical path so current measurement represents the complete vehicle rather than one isolated branch.

After whole-vehicle current measurement, the power system is divided into two 5 V buck-converted branches. One branch is dedicated to motor power, while the other supplies the Raspberry Pi, ESP32 electronics, displays, sensors, and supporting peripherals. I originally expected the vehicle could approach the current rating of a single 5 A converter, so splitting the load gave more headroom and also helped separate electrically noisy motor loads from the rest of the electronics.

The installed prototype uses a 7.5 A fuse because early current estimates were much higher than what the completed system eventually drew. Final testing showed a normal peak around 1.6 A and typical driving around 1.2 to 1.3 A. Based on those measurements, I would reduce the fuse to approximately 5 A in a future revision so the protection value better matches the real system while still leaving startup and transient margin.



Power path from 2S LiPo through fuse, main switch, INA228 current measurement, and separate motor/electronics buck-converter branches.

4.3 Decoupling and Power Stability

The final hardware includes 0.1 uF ceramic decoupling near ESP32 and motor-driver supply points. Larger 470 uF, 10 V electrolytic capacitors are used near the servo and motor-driver loads to provide local energy storage during fast current changes. All controllers share a common ground reference.

One of the most important electrical lessons came from a breadboard with split ground rails. I initially assumed the rails were internally connected, which caused inconsistent sensor behavior and stopped hardware progress for roughly two days. Once I identified the missing ground continuity and tied the rails together correctly, the sensors became stable. That failure reinforced how often embedded-system problems come from physical power or connection issues rather than software.

4.4 Current Sensor Selection

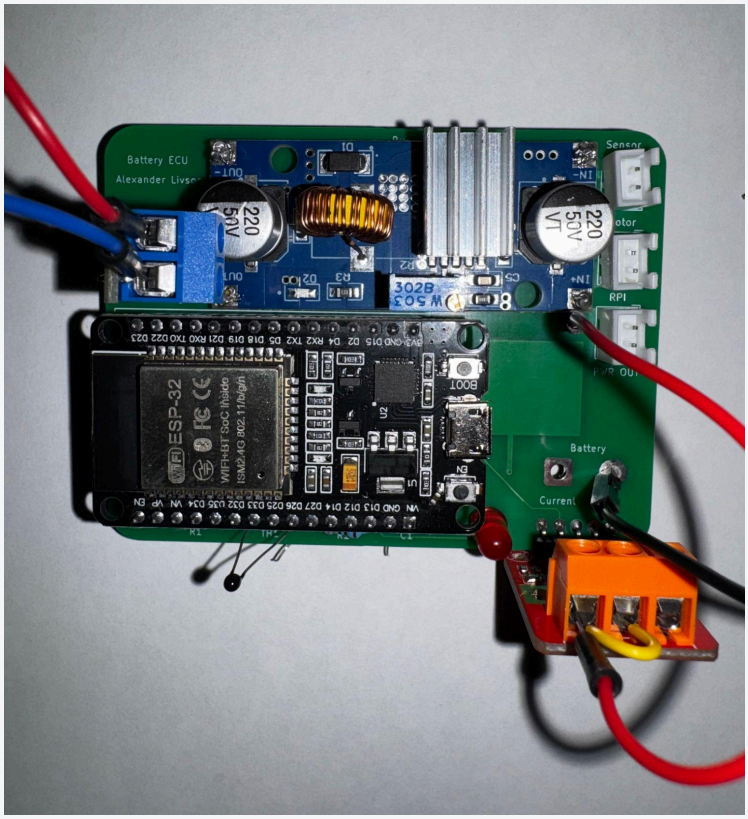
The original current-sensing approach used an INA219-class module. It was difficult to use reliably in the intended 3.3 V logic environment and its current range was less comfortable for the vehicle I was planning. I replaced it with an INA228-based module, which provided a higher-current-capable and more reliable monitoring solution while working with the 3.3 V system. The INA228 became the final whole-vehicle current and voltage monitor.

5. ECU Design

5.1 Battery ECU

The Battery ECU is dedicated to vehicle power health. Its main job is to report pack voltage, current, and temperature over CAN so the Raspberry Pi can make supervisory decisions. This controller exists separately because LiPo batteries require careful monitoring, and I wanted power health to remain a separate subsystem rather than an afterthought inside the sensor or motor code.

The Battery ECU reads the INA228 for electrical measurements and a 10 kOhm NTC thermistor for temperature. Its CAN status frame is decoded by the Raspberry Pi as millivolts, centi-degrees Celsius, and milliamps. A local LED is used as a health/status indicator.



Custom Battery ECU with ESP32 controller, CAN interface, INA228 monitoring path, thermistor input, power conversion, and status indication.

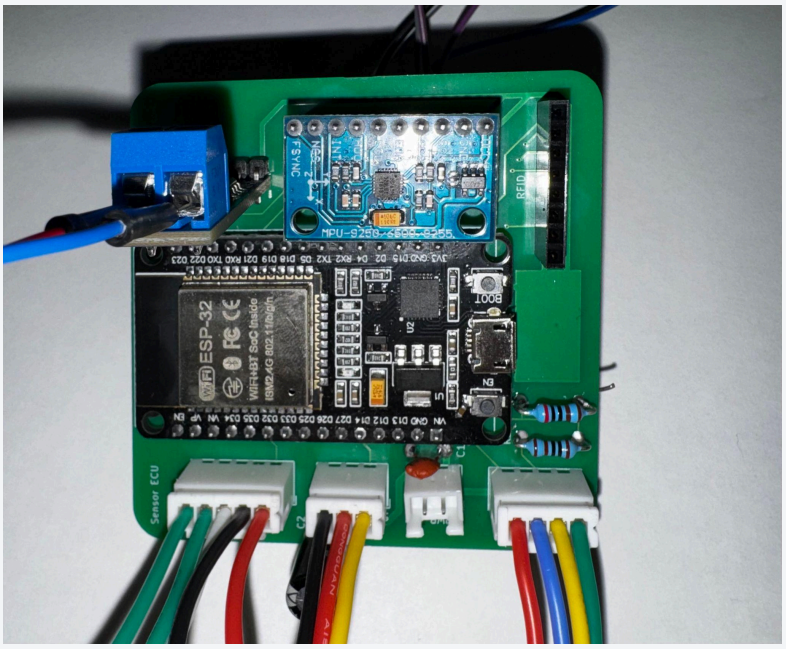
5.2 Sensor ECU

The Sensor ECU became the most peripheral-dense board in the vehicle. It handles the rotating HC-SR04 ultrasonic sensor, microservo, MPU9250/MPU6500-class IMU, MFRC522 RFID reader, two OLED displays, and MAX7219 LED matrix. Grouping these functions on one ECU kept the sensing and human-interface hardware localized and prevented the Raspberry Pi from becoming a large GPIO breakout board.

The ultrasonic sensor scans approximately 60 degrees left and right, sampling left, center, and right directions as the servo moves through its pattern. These measurements are reported over CAN and are used both for the dashboard radar view and as a fallback direction source during autonomous obstacle avoidance.

The two OLED displays provide on-vehicle information. One focuses on vehicle state and drive mode, while the other shows battery information such as estimated percentage, voltage, current, and temperature. The LED matrix provides directional feedback with forward, reverse, left, and right arrow patterns.

The RFID reader functions like a simplified vehicle key. The vehicle starts locked, and an authorized RFID scan changes the Sensor ECU's lock state so motor movement can be enabled. The browser can also request lock or unlock for debugging, but the Sensor ECU remains authoritative. This prevents the Raspberry Pi from assuming that a requested state change succeeded. Instead, the Sensor ECU applies the change and reports the confirmed lock state back over CAN.

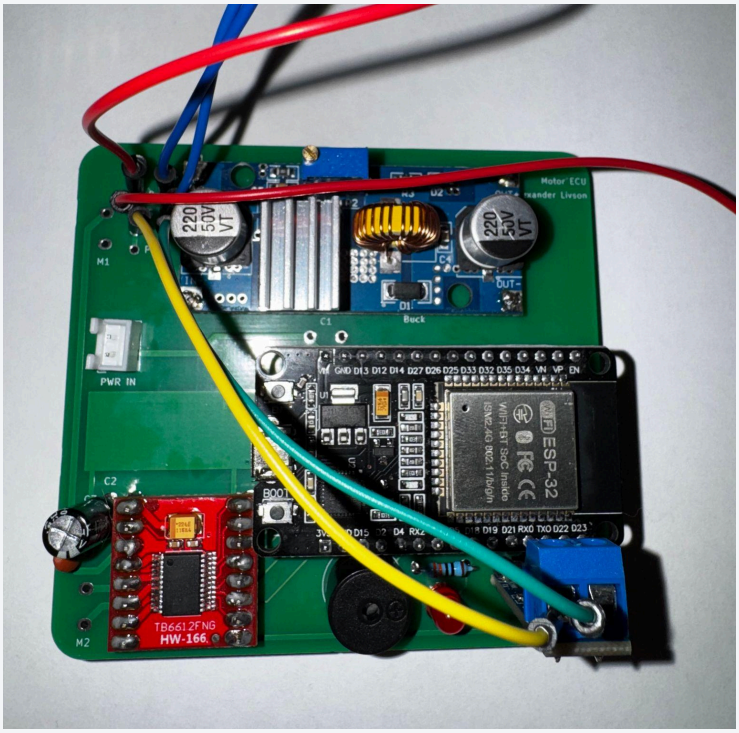


Sensor ECU integrating ultrasonic/servo scanning, IMU, RFID, dual OLED interfaces, LED matrix, and CAN communication.

5.3 Motor ECU

The Motor ECU converts high-level left/right drive commands into actual differential motor control. I selected the TB6612FNG because it is substantially more efficient and better suited to this small vehicle than the older L298N-style driver that came with many hobby kits. The two TT motors are controlled independently, which allows forward/reverse motion and tank-style turning.

Motor commands are expressed as signed percentages from -100 to +100 and are limited by the active speed limit. The Motor ECU reports left/right output feedback, enabled status, and room for fault flags through CAN ID 0x300. Local LED and buzzer outputs provide health and fault indication, including emergency and overcurrent conditions.



Motor ECU with ESP32, CAN interface, TB6612FNG dual motor driver, status LED, buzzer, and motor connectors.

6. CAN Communication Architecture

6.1 Physical Bus

The completed vehicle uses a four-node CAN bus operating at 500 kbps. The Battery ECU and Raspberry Pi are physically located at the two ends of the bus. Termination is provided at those endpoints, producing an approximately 60 ohm measurement across CAN-H and CAN-L with the system powered off. That measurement was used as a quick validation that two 120 ohm terminations were effectively present.

The Pi uses Linux SocketCAN through interface can0. A systemd service configures and enables the CAN interface at boot so the Python controller can open the already-configured bus. On the embedded side, each ESP32 uses a 3.3 V-compatible CAN transceiver module.

6.2 Message Organization

I defined the project CAN protocol so each message has a specific subsystem owner and purpose. Battery telemetry uses the 0x100 range, sensor telemetry uses the 0x200 range, Motor ECU status uses 0x300, and Raspberry Pi supervisory commands use 0x400. This makes bus traffic easy to recognize in candump and keeps the protocol organized as the project grows.

CAN ID	Direction	Purpose
0x100	Battery ECU -> Pi	Battery voltage, temperature, current
0x200	Sensor ECU -> Pi	Forward range, servo angle, RFID/lock state, sensor-health flags
0x201	Sensor ECU -> Pi	Accelerometer X/Y/Z
0x202	Sensor ECU -> Pi	Gyroscope X/Y/Z
0x203	Sensor ECU -> Pi	Roll, pitch, yaw
0x204	Sensor ECU -> Pi	Left, center, right ultrasonic scan ranges
0x300	Motor ECU -> Pi	Actual left/right motor output, enabled state, fault field
0x400	Pi -> vehicle	Safety-filtered left/right drive command, mode, speed limit, system flags
0x401	Pi -> Sensor ECU	Browser-originated lock/unlock request

6.3 Supervisory Timing

The Raspberry Pi control loop runs every 20 ms, corresponding to a nominal 50 Hz supervisory update rate. The Pi transmits the final safety-filtered drive command from that loop. Manual and Assisted browser control use a 150 ms heartbeat to keep the Pi's control watchdog alive, while actual key transitions are transmitted immediately through the web API rather than waiting for the next heartbeat.

The Pi uses a 500 ms browser-control timeout and a 500 ms critical-ECU telemetry timeout. Battery and Sensor ECU data older than this threshold prevents motion. The 500 ms value leaves space for brief scheduling delays or a dropped frame while still ensuring the vehicle does not continue driving for long without fresh information. Motor ECU status is also monitored and displayed; in the current Pi implementation, stale Battery and Sensor telemetry are hard drive interlocks, while Motor ECU staleness is primarily a health/diagnostic indication.

6.4 CAN Debugging

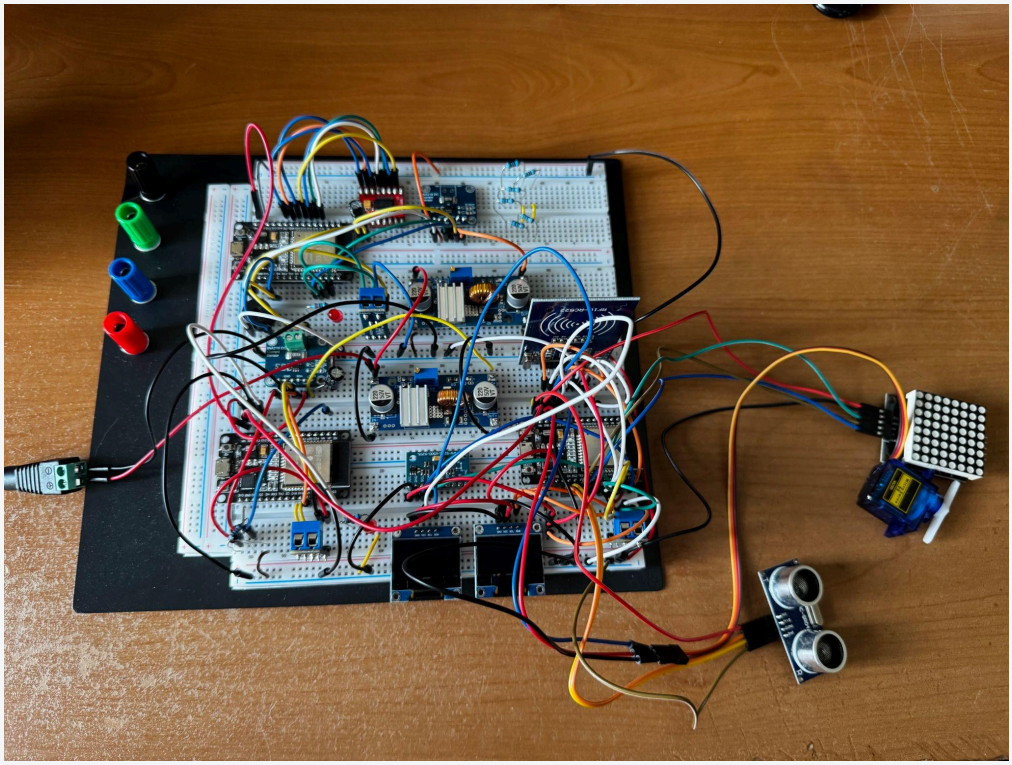
CAN bring-up was one of the most useful parts of the project because the failures were physical as often as they were software-related. I used candump, ip link, terminal editing, SSH, and Linux device inspection repeatedly while testing the network. One major issue came from pin assignments that had changed between breadboarding and PCB design. Some CAN and I2C pins were not updated consistently, and using ESP32 serial RX/TX-related pins created boot problems. The boards were still recoverable through firmware changes and wiring corrections, but the experience showed why final pin mapping needs to be frozen before layout.

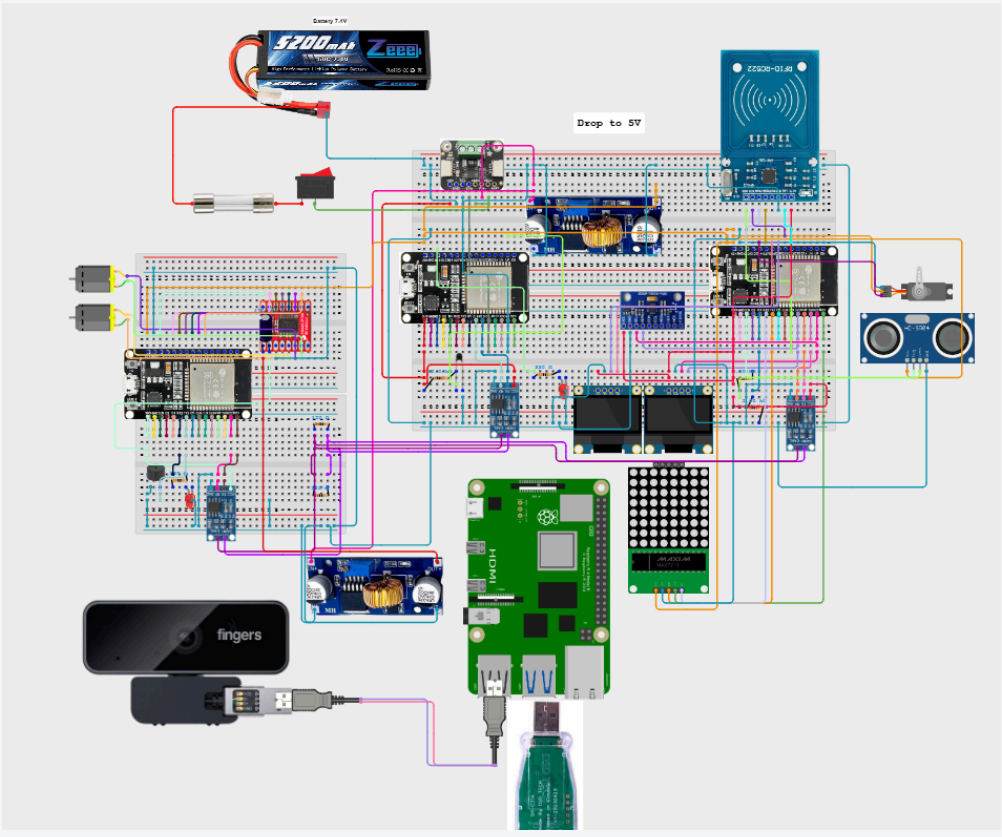
7. Custom PCB Development

7.1 From Breadboard to KiCad

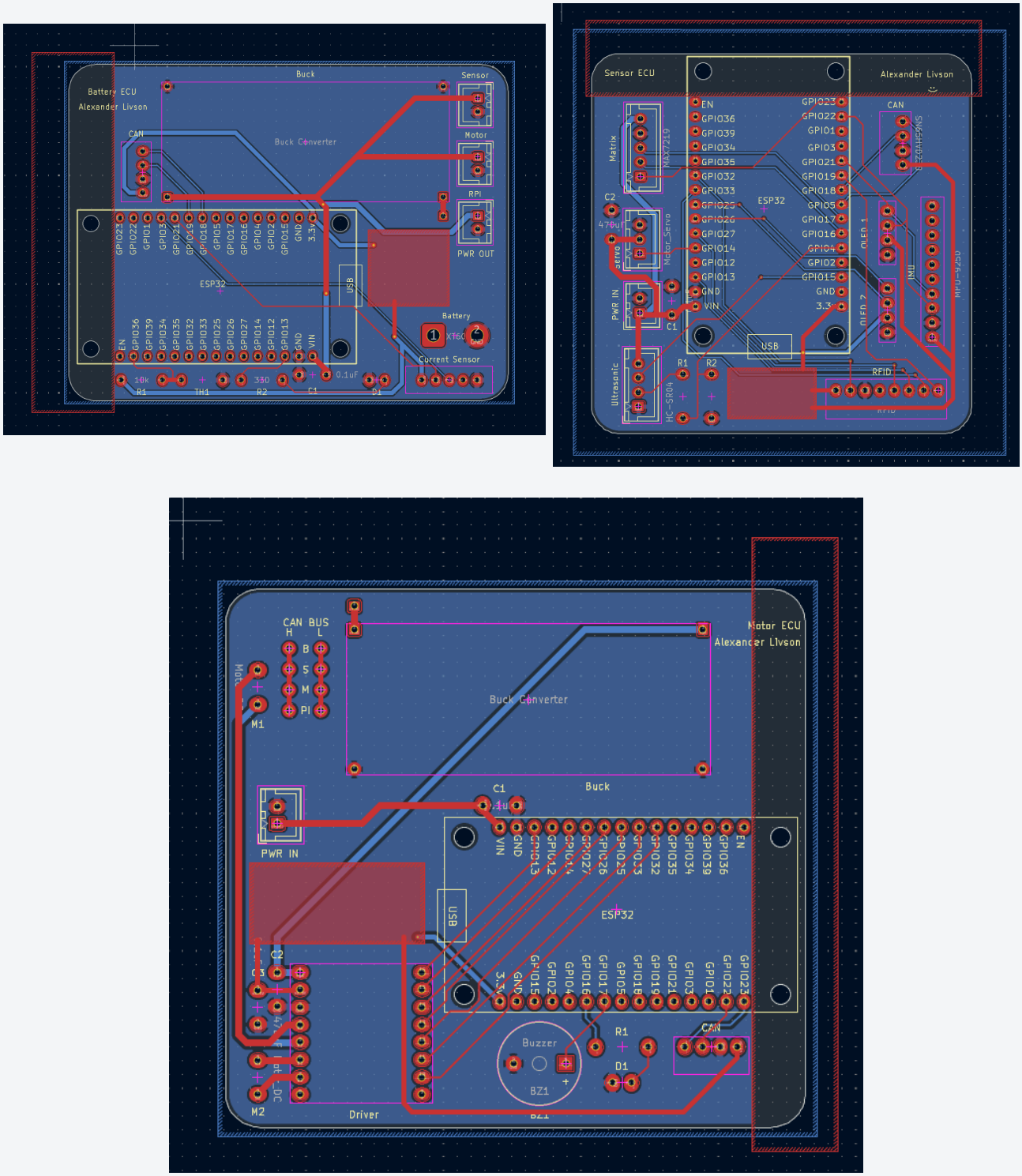
PCB design was the main technical reason I started the project. Before this, I had very little experience and had previously tried Altium Designer without getting comfortable with the workflow. For this project I learned KiCad and translated each validated breadboard subsystem into a schematic and then into a manufacturable PCB.

All three ECU PCBs were designed by me. The process took roughly two focused days for the first complete set of boards once the breadboard designs were stable. Because I intentionally used complete sensor and controller modules rather than bare ICs, many available library footprints were not appropriate. I created a large number of custom through-hole footprints for development boards, breakout modules, connectors, displays, and sensors, especially where existing footprints were SMD-only or had different mechanical spacing.





Breadboard validation stage for Battery, Sensor, and Motor subsystems before schematic capture.

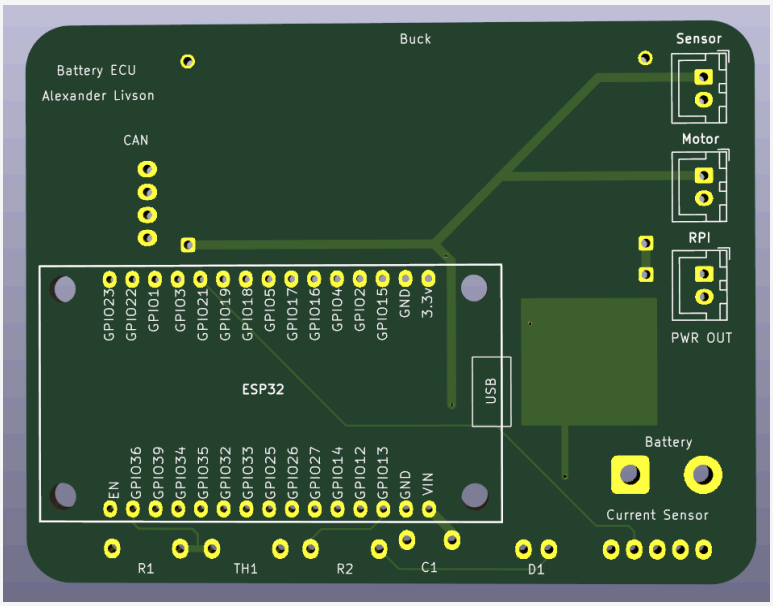


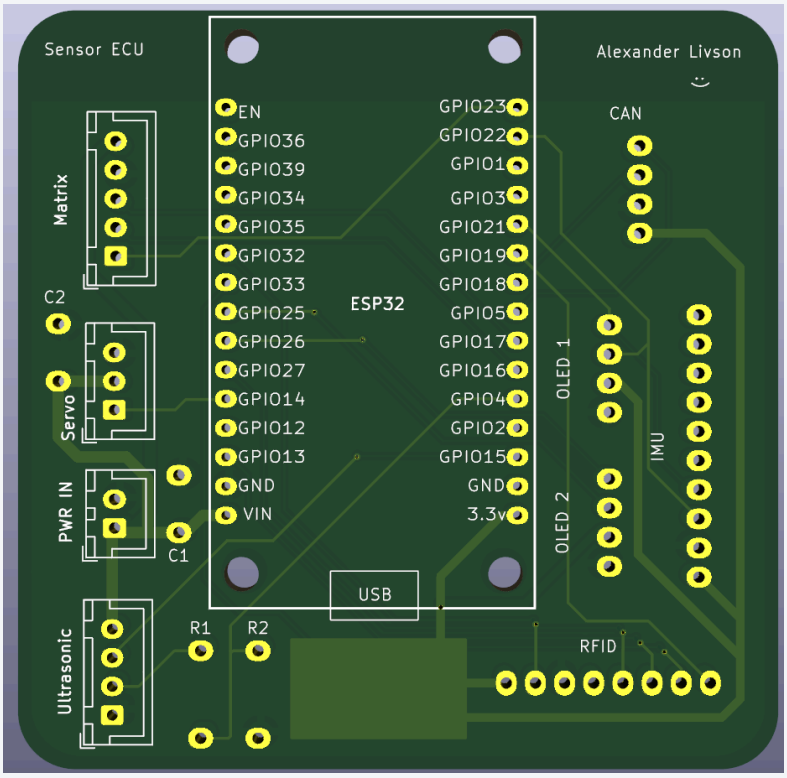
Final Battery, Sensor, and Motor ECU schematics translated from validated breadboard circuits.

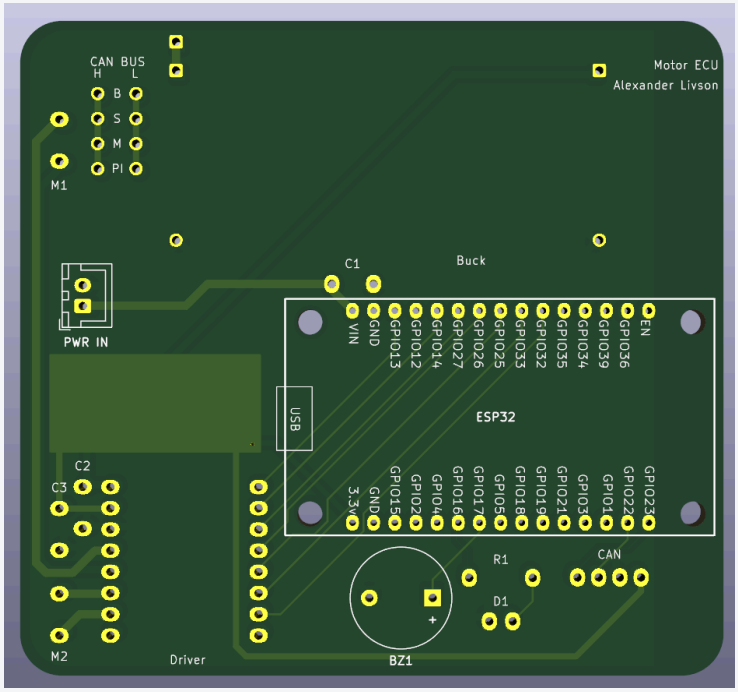
7.2 Layout Strategy

The boards are two-layer designs. I used the bottom layer primarily as a ground plane and routed signals and power on the top where practical. Typical signal traces were approximately 0.25 mm, 3.3 V distribution was around 0.75 mm, and 5 V/high-current paths were generally around 1.0 mm. Board thickness was approximately 1.6 mm.

I also removed copper beneath the ESP32 antenna area to reduce the chance of the PCB interfering with wireless performance. All three boards passed KiCad DRC before fabrication.







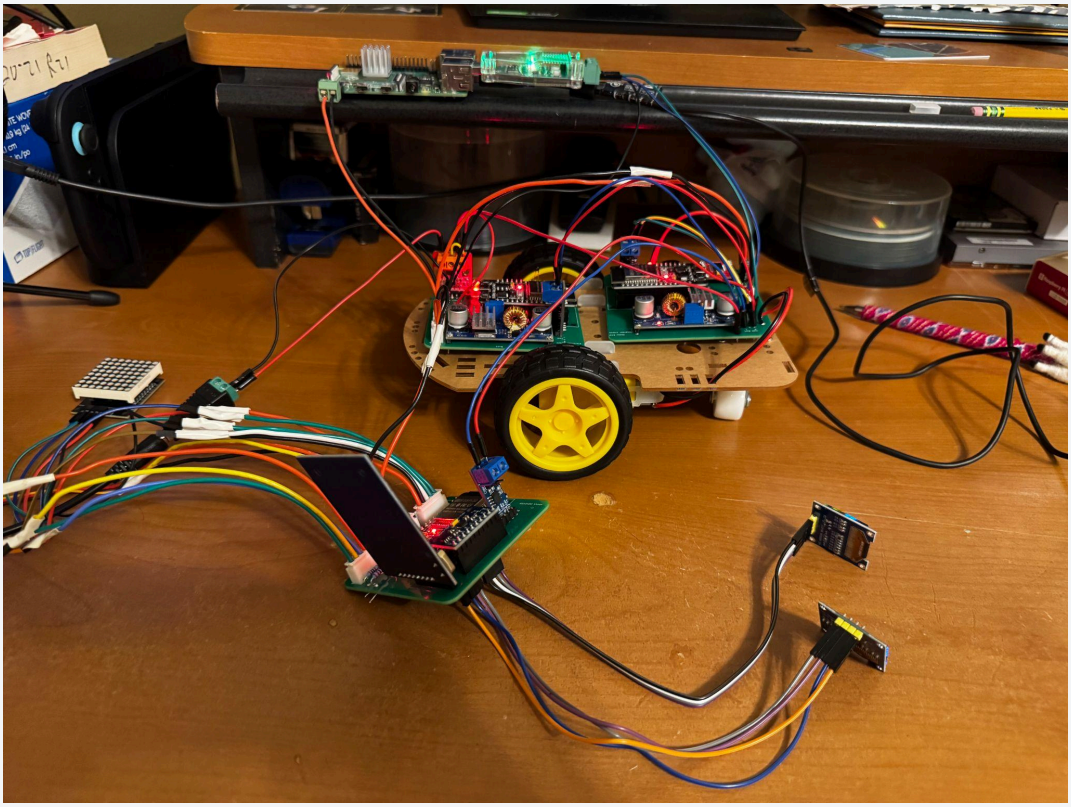
Final two-layer PCB layouts showing power routing, connectors, and ESP32 antenna keep-out.

7.3 Fabrication and Assembly

The boards were manufactured by JLCPCB. I hand soldered the three final vehicle boards, which took approximately seven to eight hours total.

The main assembly issue was not a manufacturing defect, but my own soldering. Several joints were cold or did not have enough solder, especially in crowded areas around larger modules and connectors. Reflowing and adding solder corrected the faults. The Sensor ECU OLED wiring also had to be routed from the underside because I had misjudged physical clearance in the original layout.

Those failures were useful because they made PCB bring-up much more real than simply receiving a perfect board. By the end of the project I was much more comfortable distinguishing a schematic/layout problem from a solder-joint problem and much more confident hand soldering dense through-hole module boards.



Completed custom ECU set after soldering, rework, and hardware validation.

7.4 Revision 2 PCB Changes

- Improve spacing around large modules and connectors to make soldering and service access easier.
- Clean up CAN routing and freeze all pin assignments before final layout.
- Reduce overall package size and move toward more compact components or selective SMD construction.
- Retain the JST-based modular connection approach, which worked well during integration.
- Keep the proven decoupling and local bulk capacitance strategy.

8. Raspberry Pi Supervisory Software

8.1 Software Stack

The Raspberry Pi 4 runs Raspberry Pi OS Lite. I used Python for the vehicle controller because it provided a practical integration point for SocketCAN, OpenCV, threading, networking, JSON APIs, and rapid development on the Pi. Flask was used for the local dashboard because it is lightweight and made it straightforward to expose control endpoints and live telemetry without adding a large application framework.

The Pi controller runs multiple concurrent tasks: SocketCAN receive handling, a 50 Hz control loop, shared camera capture, OpenCV analysis, Flask requests, and dashboard state updates.

8.2 Boot and Service Behavior

The vehicle is configured to bring up CAN automatically during boot with a systemd service. The main Python controller also auto-starts. Practical boot-to-ready time is roughly 20 seconds, with CAN initialization accounting for a noticeable portion of that startup.

If CAN is not available, the application reports the error and prevents the vehicle from fully functioning. The web interface displays ECU health and other system warnings so a communication problem is visible before the vehicle is driven.

8.3 Command Freshness and Browser Watchdog

The browser control path includes two protections against delayed or stale requests. Every dashboard load receives a new client-start timestamp, and every drive command from that page uses an increasing sequence number. The Pi rejects commands from an older dashboard session and rejects duplicate or out-of-order sequence numbers from the active page.

This was added while debugging a serious control-latency problem. It ensures that an old delayed FORWARD request can never arrive after a newer STOP request and regain control. The browser also sends key transitions immediately, while a slower background heartbeat only exists to keep the 0.5 s control watchdog alive. If the browser stops sending fresh commands, the requested motor outputs are set to zero.

8.4 Dashboard State Refresh

The browser polls vehicle state after each previous request finishes, with a nominal 250 ms delay between refreshes. This avoids building a queue of overlapping state requests if the local network briefly slows down. The resulting dashboard update rate is approximately 4 Hz, while the actual vehicle control loop remains much faster at 50 Hz.

9. Drive Modes and Autonomous Behavior

9.1 Manual Mode

Manual mode gives the driver direct WASD control through the browser. A speed slider limits the maximum signed motor command from 0 to 100%. Because the vehicle uses differential steering, forward/reverse and turn requests are converted into left and right motor percentages.

Manual mode is intentionally not completely unprotected. A hard forward collision rule remains active so the vehicle will not continue driving into a very close obstacle if the operator loses control or cannot reach the power switch. Reverse motion remains available when an obstacle is detected in front so the driver can back away.

9.2 Assisted Mode

Assisted mode keeps the same manual WASD interface but adds distance-based intervention. At 40 cm the system begins linearly reducing positive motor commands. At 20 cm, positive forward commands are removed. Below 12 cm, the safety manager forces a complete stop for forward-requested motion. This gives the driver control while adding a second supervisory layer between the requested command and the motors.

Forward Distance	Assisted Response
> 40 cm	Driver command passes through, subject to other safety limits
20-40 cm	Forward output scales down linearly as the wall gets closer
12-20 cm	Positive/forward motor command removed
< 12 cm	Hard wall intervention, drive command forced to zero for forward-requested motion

9.3 Autonomous Mode

Autonomous mode began as a simpler ultrasonic obstacle-avoidance routine. The vehicle would drive forward, slow near a wall, and use the servo scan to compare left and right free space. That approach worked, but I wanted the camera to contribute useful information instead of being only a dashboard stream.

The final version is best described as vision-assisted obstacle avoidance. The ultrasonic sensor remains the authority for actual obstacle proximity because it gives a direct distance measurement. OpenCV is used only to decide which side of the camera view appears less obstructed. Keeping those responsibilities separate

avoids treating a visually simple surface, such as a blank wall, as open space just because it has fewer visual features.

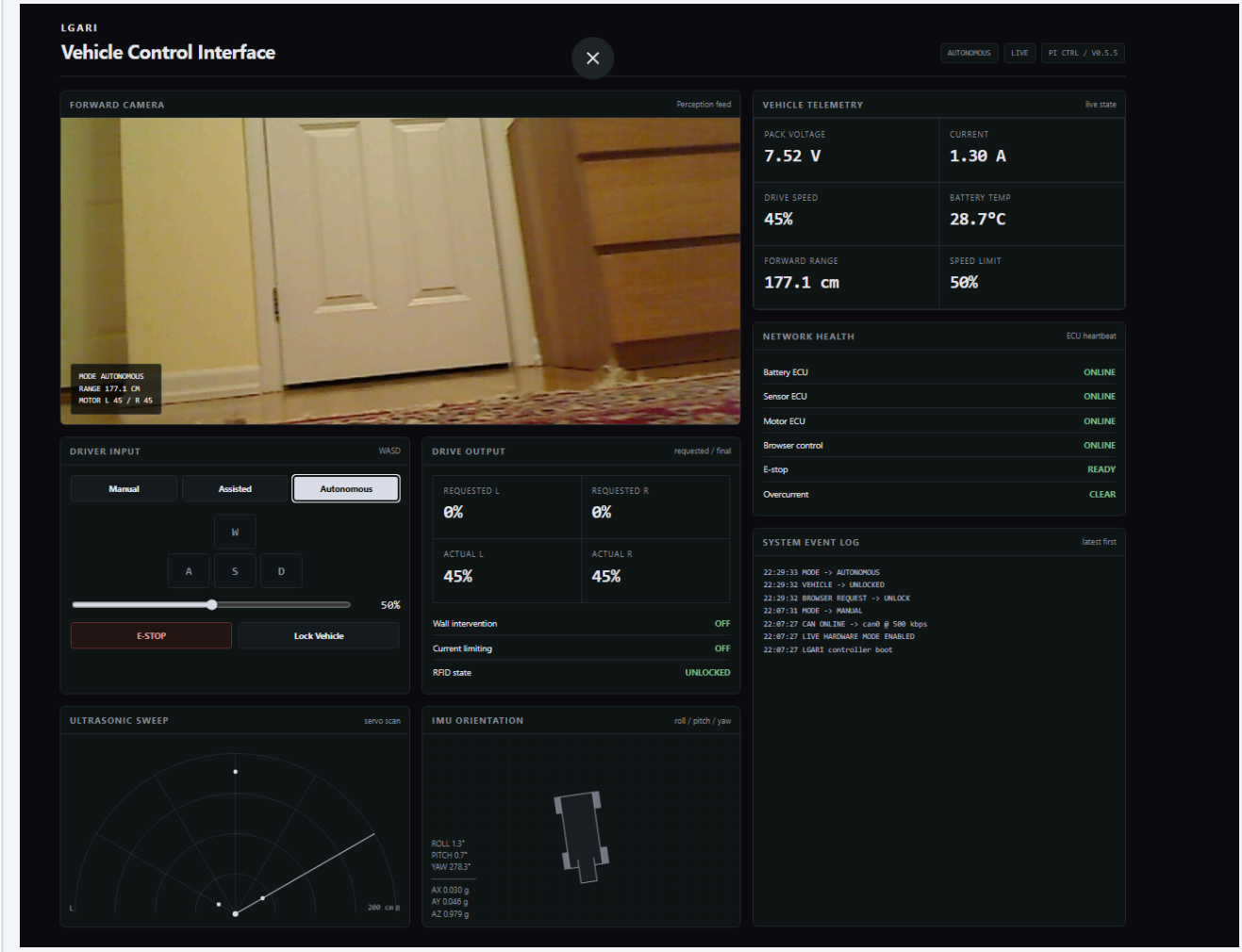
OpenCV Obstruction Scoring

The camera frame is cropped to the useful lower portion of the scene, converted to grayscale, blurred to reduce noise, and processed with Canny edge detection. The edges are slightly dilated so clusters of visual structure count as occupied regions rather than isolated pixels. The image is then divided into left and right regions. Edge density is calculated for each side, with the lower half weighted more heavily because it is more likely to represent objects in the vehicle's path.

If the left obstruction score is higher, the vehicle turns right. If the right score is higher, it turns left. The software only uses camera scoring when the frame is recent and the difference between the two sides is large enough to be meaningful. If the camera is unavailable or the result is nearly tied, the vehicle falls back to the existing ultrasonic left/right scan.

Autonomous Recovery Sequence

1. Drive forward at up to 45% command when the forward range is greater than 50 cm.
2. Reduce autonomous cruise to approximately half speed between 50 cm and the 32 cm recovery trigger.
3. When an obstacle reaches approximately 32 cm, select a turn direction using OpenCV if a recent decisive camera result is available, otherwise use the ultrasonic scan.
4. Back straight up at up to 25% command for approximately 0.35 s.
5. Turn in place at up to 38% command for approximately 0.55 s.
6. Return to forward driving and repeat the process when another obstacle is encountered.



Forward camera feed used by the Raspberry Pi for vision-assisted left/right obstruction analysis.

10. Safety Architecture

Safety was treated as a separate supervisory layer rather than something each drive mode could optionally use. The main safety function receives requested left/right motor outputs and has the authority to modify or completely reject them before the Pi transmits CAN ID 0x400.

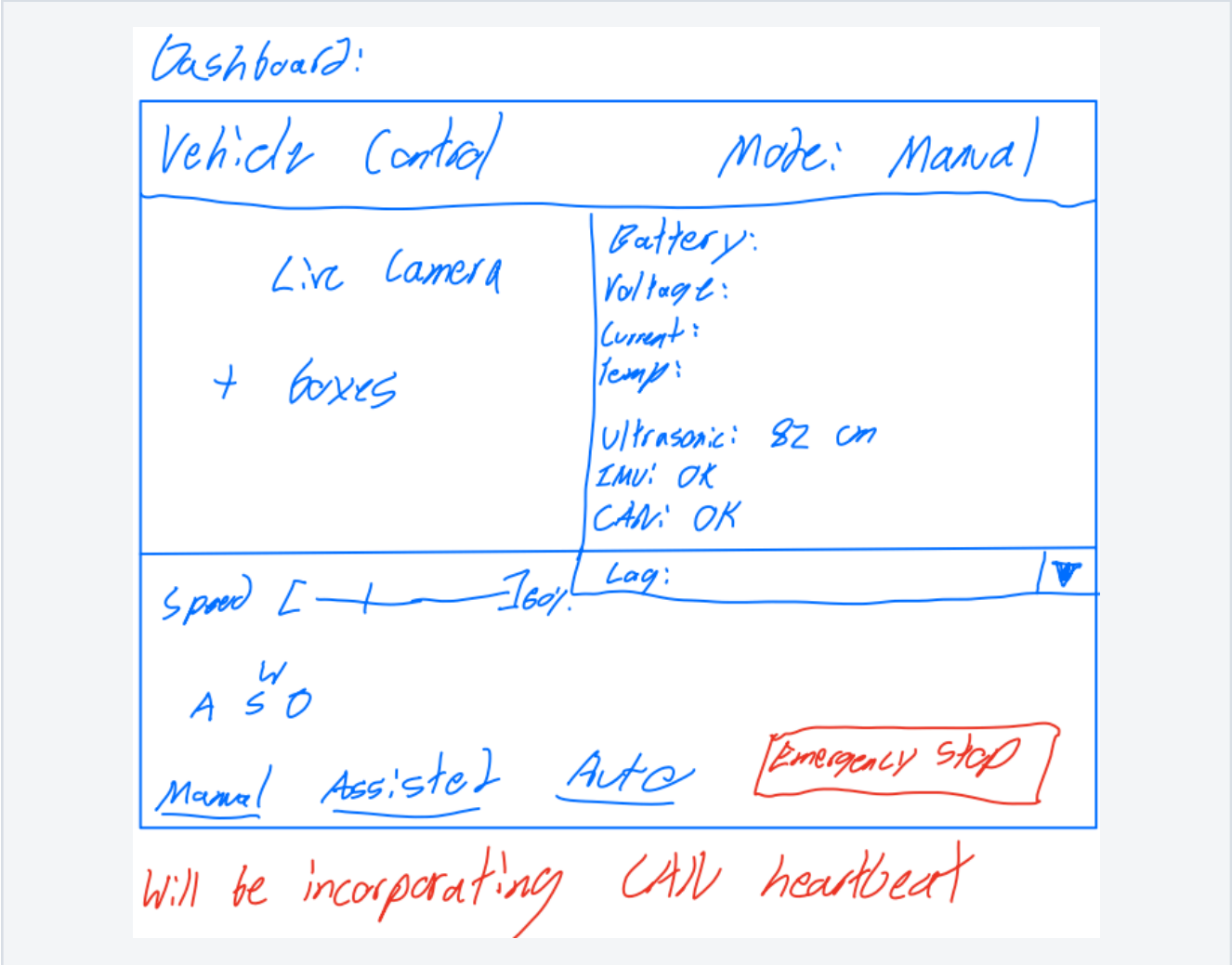
Safety Function	Final Behavior
E-stop	Overrides drive requests and commands zero motor output.
RFID lock	Vehicle starts locked; motor movement is blocked until Sensor ECU reports an authorized/unlocked state.
Browser watchdog	Manual/Assisted input expires after 0.50 s without a fresh control request.
Battery ECU freshness	Motion blocked if Battery ECU telemetry is older than 0.50 s.
Sensor ECU freshness	Motion blocked if Sensor ECU telemetry is older than 0.50 s.
Wall intervention	Manual hard stop; Assisted/Auto slow-down and stop behavior based on 40/20/12 cm thresholds.
Current warning	> 4.75 A reduces commanded outputs to 75% of their requested value.
Current limit	> 5.25 A reduces commanded outputs to 45%.
Overcurrent shutdown	> 5.50 A commands zero output and latches a 1.5 s cooldown.
Motor ECU health	Motor status and feedback monitored on dashboard; current Pi code does not use Motor telemetry staleness as the same hard interlock applied to Battery/Sensor data.

The current thresholds were chosen around the expected limits of the 5 A power-converter branches. The completed vehicle draws far less current than initially predicted, so these thresholds were not reached during normal operation, but the logic remains useful as a supervisory protection layer and as a framework for future higher-power versions.

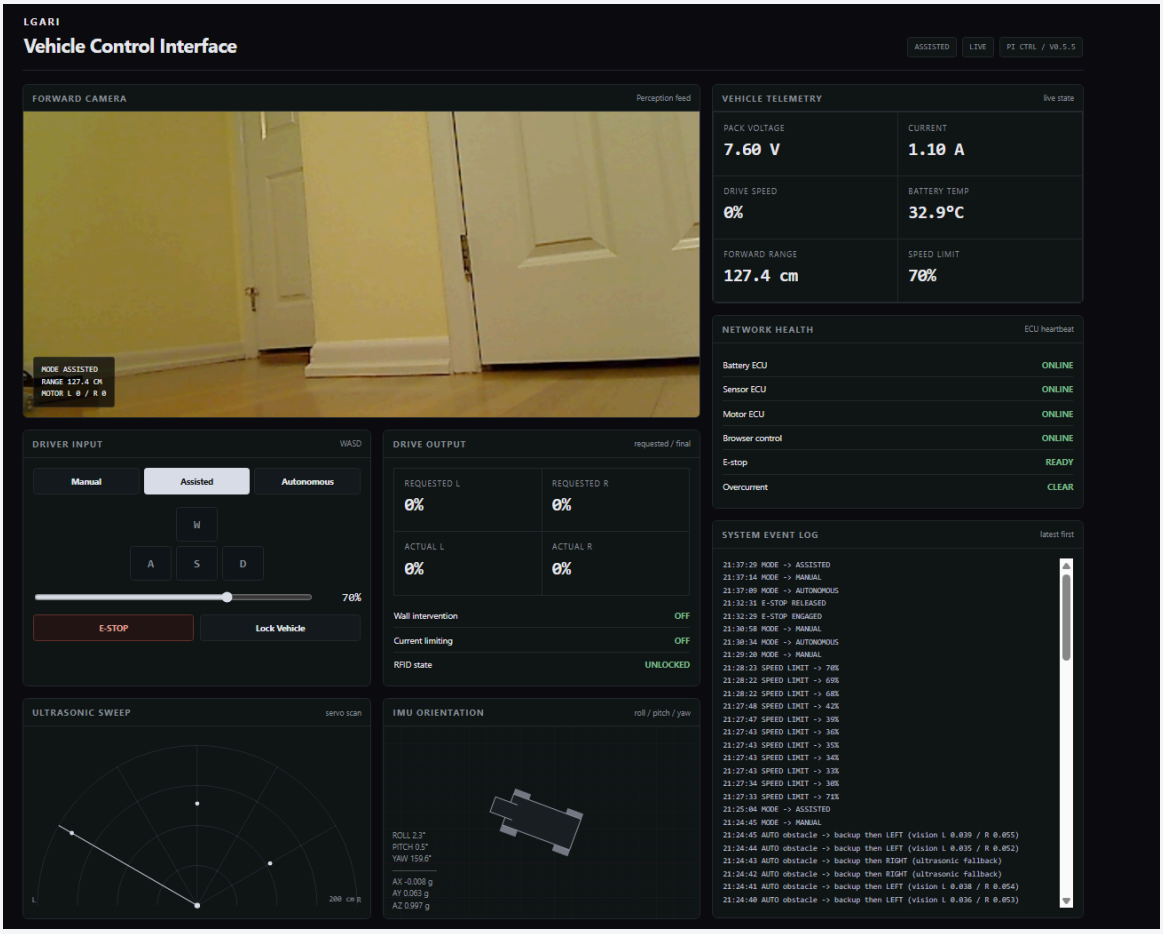
The underlying principle is fail-safe behavior. When the software does not have enough trustworthy information to know that a motion command is safe, the preferred state is no motion. This is especially important in a distributed system because stale data can look valid unless freshness is explicitly tracked.

11. Web Dashboard and Human-Machine Interface

The Raspberry Pi hosts a local browser dashboard that acts as the main human-machine interface. The original dashboard was first planned as a handwritten sketch and then iterated into a much more complete live interface. The final layout emphasizes the forward camera, vehicle mode, driver controls, motor outputs, battery telemetry, ultrasonic range, IMU visualization, network health, safety status, and event history.



Initial dashboard concept before implementation.



Final browser-based vehicle control interface.

11.1 Driver Controls

- Manual, Assisted, and Autonomous mode selection.
- WASD differential-drive control for Manual and Assisted modes.
- 0-100% speed-limit slider.
- Emergency-stop control.
- Browser lock/unlock request for debugging.

11.2 Telemetry and Diagnostics

- Pack voltage, current, and battery temperature.
- Forward ultrasonic distance and servo-scan radar visualization.
- Requested and Motor ECU-reported left/right outputs.
- IMU roll, pitch, yaw, and acceleration values.
- Battery, Sensor, and Motor ECU online status.
- Browser-control watchdog status, E-stop state, overcurrent state, wall intervention, and current limiting.
- Event log for mode changes, speed changes, lock state, faults, and other system events.

The dashboard's Drive Speed field is intentionally based on the average magnitude of the Motor ECU output percentages because the vehicle does not have wheel encoders. It should therefore be interpreted as drive effort rather than measured road speed. A future revision with encoders would allow true velocity and odometry.

One UI improvement I would make is to make battery percentage more prominent in the web interface. Battery data is visible on the vehicle itself, but the current browser dashboard emphasizes voltage, current, and temperature rather than presenting a dedicated large percentage value.

12. Integration, Debugging, and Engineering Challenges

12.1 Split Ground Rails

The longest single hardware debugging issue came from split breadboard ground rails. I assumed the rails were connected and spent roughly two days investigating unstable sensor behavior. The eventual fix was that the grounds were not continuous. This was frustrating, but it became one of the most useful lessons from the project because it changed the order in which I debug hardware. I now verify power, ground, continuity, and physical connections before assuming a sensor or firmware library is the problem.

12.2 PCB Solder-Joint Failures

After PCB assembly, several peripherals initially behaved inconsistently because of cold joints and insufficient solder. Rather than assuming the fabricated boards were defective, I inspected and reworked the connections. Adding solder and reheating the affected joints restored operation. This improved both my soldering technique and my confidence doing board bring-up.

12.3 Pin-Mapping Errors

Some pin assignments changed during breadboard development but were not frozen before PCB layout. This produced CAN and I2C mapping issues on the manufactured boards, including use of ESP32 serial-related pins that interfered with boot behavior. The system was still recoverable through software and wiring adjustments. For a future design, I would treat the pin map as a controlled interface document before schematic completion.

12.4 Servo Jitter and Peripheral Reality

The ultrasonic servo initially jittered because the control approach assumed more precise behavior than the inexpensive hobby servo could consistently provide. Adjusting the sweep behavior to match the real hardware eliminated the practical issue. That was a useful reminder that software should be designed around measured component behavior rather than idealized specifications.

12.5 Network Control Latency

The most significant software/networking problem was a repeatable multi-second delay between browser commands and motor response. At its worst, typical control behavior could lag by roughly three to five seconds. Software instrumentation showed command delays on the order of hundreds of milliseconds even after several improvements.

I first hardened the browser control path so immediate key transitions could bypass a slow background heartbeat. I also added dashboard-session timestamps and sequence numbers so delayed or out-of-order requests could never overwrite a newer command. Those changes improved consistency and reduced the remaining visible lag to roughly half a second, but did not remove the root cause.

The final breakthrough came from accessing the Raspberry Pi directly through its IPv4 address instead of the local hostname. In this network setup, hostname access was resolving through an IPv6 path that produced the large delay. Direct IPv4 access reduced practical control latency to approximately 20-30 ms and made the vehicle feel immediate. The debugging process was valuable because it separated vehicle-control latency from network-resolution latency and left the software with stronger stale-command protection even after the network issue was solved.

12.6 Audible Buck-Converter Noise

One buck converter produced noticeable audible switching noise during testing. The system remained functional, but the behavior showed another practical difference between a schematic and real power hardware. In a more polished revision I would select power converters specifically for electrical performance, acoustic behavior, efficiency, and transient response rather than using generic modules.

13. Testing and Validation

13.1 Measured Final-System Values

Metric	Observed / Final Value
Boot-to-ready time	Approximately 20 s
Idle current	Approximately 0.8-0.9 A
Typical driving current	Approximately 1.2-1.3 A
Peak observed current	Approximately 1.6 A
Battery runtime	Approximately 1 hour
Direct IPv4 control latency	Approximately 20-30 ms, not noticeable in normal driving
RFID operating distance	Very short range, within a few centimeters
Ultrasonic usable distance	Approximately 160 cm in practical testing
CAN bitrate	500 kbps
Pi supervisory loop	50 Hz
OpenCV scoring interval	Nominal 10 Hz software interval
Dashboard state refresh	Nominal 4 Hz
Browser drive heartbeat	Approximately 6.7 Hz
Normal test speed limit	Typically 70%; system also tested at 100%
Estimated maximum speed	Approximately 4 ft/s (about 1.2 m/s), not encoder-verified
Dimensions	24 x 19 x 19 cm (L x W x H)
Mass	Approximately 872 g including battery
Approximate cost	$300

13.2 Functional Validation

The final integrated vehicle was validated as a complete system rather than only as separate boards. The three ECUs could be powered from the vehicle, CAN traffic was received by the Raspberry Pi, the dashboard reported live data, RFID state controlled drive permission, Manual and Assisted modes responded to browser input, and autonomous obstacle avoidance operated with the camera and ultrasonic sensor.

Assisted collision protection was tested at multiple drive commands and reliably prevented the vehicle from continuing forward into an obstacle. Autonomous obstacle avoidance did not show failures during the informal test runs performed, although I did not run a formal counted statistical trial set. The result should therefore be described as successful functional validation rather than a quantified reliability percentage.

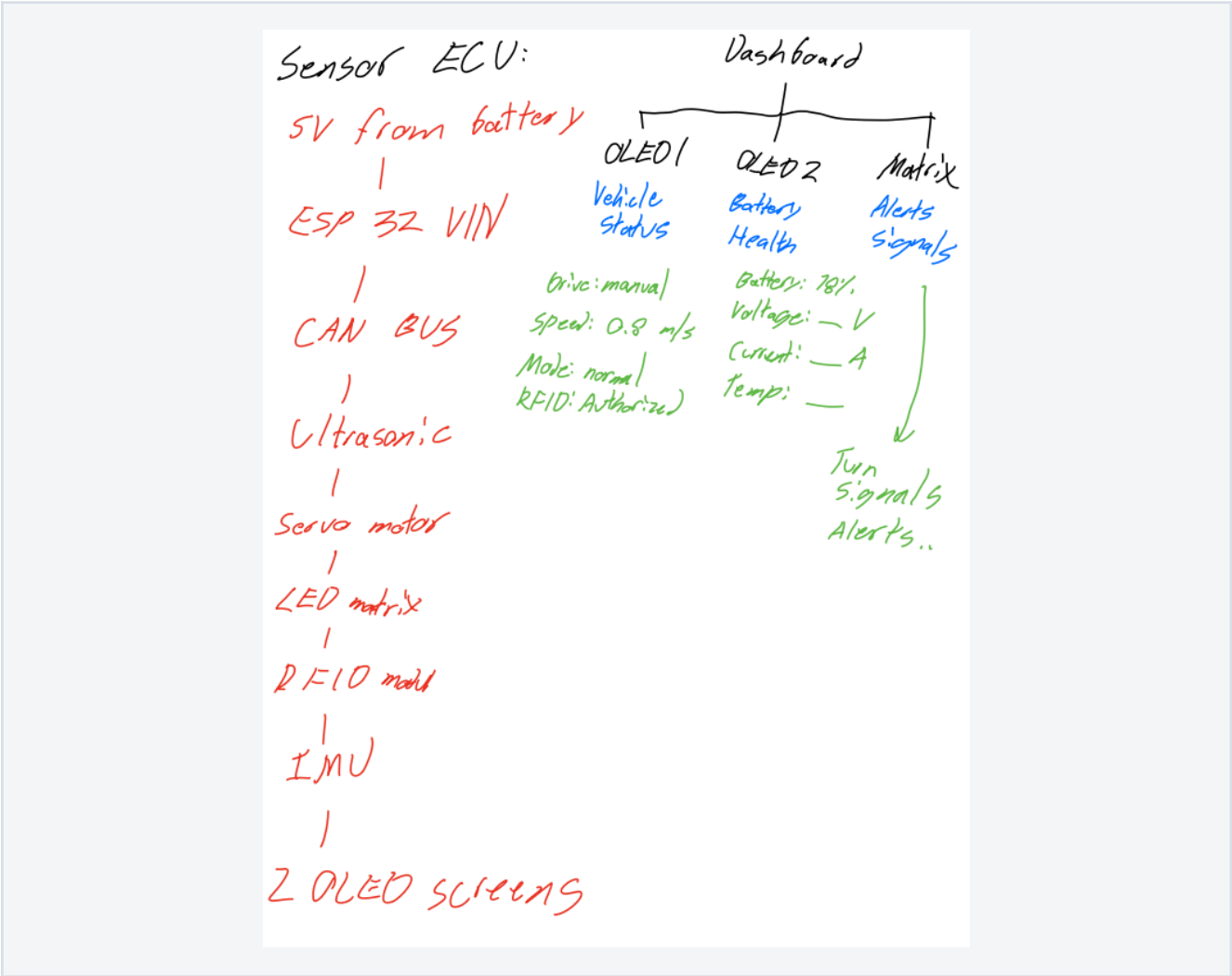
The project also validated the electrical current assumptions. Early design estimates considered currents as high as 6-7 A, but the assembled vehicle remained below about 2 A in observed operation. That difference is useful design feedback for a future fuse and power-converter selection.

13.3 Camera Performance

The final software does not explicitly force a camera resolution or frame rate, so the USB webcam operates at the default mode selected by OpenCV and the camera driver. The autonomy code intentionally limits visual obstruction scoring to a nominal 0.10 s interval, or about 10 Hz, which is sufficient for the low-speed obstacle-avoidance behavior. Exact camera-native resolution and capture FPS were not separately characterized.

14. Development Timeline

Date / Period	Milestone
July 11, 2026	Project started. Initial goal was a small multi-ECU vehicle system using CAN.
July 11-23	Architecture planning, component selection, early firmware, and breadboard hardware validation.
July 24	All three ECU concepts breadboarded; power/current-monitoring issues identified.
July 26-31	Sensor and Motor ECU validation, CAN communication bring-up, hardware-debugging work including split-ground issue.
August 1	Began KiCad learning and PCB design.
August 5	Completed all three ECU PCB designs and ordered boards.
August 10-13	Initial hand soldering, PCB bring-up, solder-joint rework, and final ECU firmware integration.
August 12	Final scope refined around three drive modes: Manual, Assisted, and Autonomous; Raspberry Pi dashboard completed.
August 15	Base vehicle electronics operating; prepared full chassis integration.
August 16	Integrated vehicle hardware, powered from LiPo, and diagnosed major browser-control latency issue.
August 17	Final assembled robot fully operational; direct IPv4 access resolved control latency.
August 18	Documentation and final OpenCV autonomy refinement.



Early project planning and development notes documenting the progression from concept to integrated vehicle.

15. Engineering Lessons Learned

15.1 PCB Design

PCB design was the area where I grew the most. I learned the complete workflow rather than only how to route traces: schematic capture, electrical organization, footprint selection, custom through-hole footprint creation, board-stack planning, power trace sizing, ground planes, antenna keep-outs, DRC, manufacturing files, physical clearance, solderability, and post-fabrication bring-up. I would be very comfortable starting another custom board from scratch after this project.

15.2 CAN and Distributed Systems

Before this project, CAN was mostly an abstract automotive term to me. Building the system taught me how a shared bus is physically wired, how termination affects the network, how identifiers create message ownership, how payloads need explicit scaling and byte definitions, and how important freshness and fault behavior become when controllers depend on one another.

I also learned that a distributed system only works when interfaces are treated seriously. A controller cannot assume another subsystem is healthy just because it was healthy a second ago. The Pi therefore tracks when critical telemetry was last seen and blocks motion when it becomes stale.

15.3 Embedded Debugging

The project reinforced that embedded debugging is usually a full-stack problem. A symptom that looks like code can actually be a ground connection, a cold solder joint, a module orientation, a pin conflict, a CAN termination issue, a power converter, or a network-resolution problem. I became much more systematic about separating electrical, communication, firmware, and application layers instead of changing several things at once.

15.4 Power and Battery Systems

I gained a much better understanding of how current moves through an embedded vehicle and why whole-system current measurement matters. Power-converter ratings, fuse values, motor transients, common grounds, local bulk capacitance, and decoupling all became practical design constraints instead of isolated topics. The Battery ECU is not a full commercial BMS, but it gave the project an explicit monitoring and protection layer.

15.5 Raspberry Pi and Linux

The Raspberry Pi became much more than a small computer running a Python script. I learned how to operate a headless Linux system over SSH, configure services, bring up SocketCAN, inspect interfaces, work with USB cameras, host a local web service, and diagnose network behavior. The direct-IPv4 latency fix was especially useful because it showed how the networking layer can dominate the apparent response of an otherwise fast embedded system.

15.6 Safety as a Separate Layer

One of my strongest takeaways is that safety should not be buried inside individual features. Drive modes request motion, but a separate safety manager decides whether that request is allowed. That structure made it possible to apply RFID authorization, stale-telemetry stops, collision rules, and current limits consistently across Manual, Assisted, and Autonomous operation.

16. Current Limitations and Future Revision

16.1 Current Limitations

- Autonomy is obstacle avoidance, not full scene understanding or route planning.
- The camera and ultrasonic sensor face forward; there is no rear obstacle sensor during autonomous backing.
- No wheel encoders are installed, so the system has no true speed feedback, odometry, or closed-loop distance control.
- There is no localization or map of the environment.
- OpenCV uses visual edge density rather than semantic object recognition, so scene texture and lighting can influence the camera score.
- The Battery ECU monitors the pack but is not a full cell-balancing battery-management system.
- The current web dashboard shows motor-output percentage rather than measured vehicle velocity.
- The prototype is built around development modules mounted on custom PCBs, which is excellent for serviceability but larger than an IC-level production design.

16.2 Highest-Priority Revision 2 Improvements

- **Rear obstacle sensing:** Add a rear ultrasonic or time-of-flight sensor so autonomous reversing has direct rear clearance information.
- **Wheel encoders:** Add measured wheel speed and odometry so the vehicle can report real velocity and execute more repeatable motion.
- **More advanced autonomy:** Expand perception beyond left/right obstruction scoring toward richer scene understanding and more deliberate path selection.
- **Packaging:** Reduce PCB and chassis footprint with tighter layout, improved component spacing, and selective SMD integration.
- **Battery protection refinement:** Use protection values and battery hardware more closely matched to the measured final load, including a lower fuse rating around 5 A for this power level.

If I had another month, my two biggest goals would be to make the autonomous system much more capable and make the entire electrical package smaller and more space efficient.

17. Conclusion

This project started as an attempt to make a few ECUs communicate and became a complete small-scale electric vehicle electronics platform. The most important part is not that the chassis can drive. The real project is the embedded system around it: custom PCBs, four-node CAN communication, power monitoring, sensor integration, a supervisory computer, safety logic, a web interface, and perception that all have to work together before the motors are allowed to move.

The final result is intentionally a simplified version of the electronics architecture found in a much larger electric vehicle. Battery health is its own subsystem. Sensors have a dedicated controller. Motor actuation is isolated. A vehicle computer combines network data and applies supervisory logic. CAN is the backbone between them. Failures are reported, stale information blocks motion, and user or autonomous requests pass through a safety layer before becoming motor commands.

The part I am most proud of is the PCB system. Before this project I had very little PCB design experience. By the end, I had designed and assembled three custom boards and integrated them into a working vehicle. Seeing all three ECUs validated was the point where the project felt real, and powering the complete vehicle while controlling and monitoring it through the dashboard was the point where the separate pieces finally became one system.

The project also changed the way I approach engineering problems. I became more comfortable moving between circuit behavior, soldering, firmware, communication protocols, Linux, networking, and application

software. More importantly, I learned that a complex embedded system becomes manageable when responsibilities are separated, interfaces are explicit, and safety behavior is designed from the beginning.

Appendix A. Final System Specifications

Category	Specification
Project period	July 11 - August 17, 2026
Architecture	3 ESP32 ECUs + Raspberry Pi vehicle computer
Network	4-node CAN, 500 kbps
Battery	2S LiPo, 7.4 V nominal, 2000 mAh, 25C
Nominal battery energy	Approximately 14.8 Wh
Installed fuse	7.5 A; future revision target approximately 5 A based on measured load
Main switch	Circular approximately 6 A switch
Power rails	Two 5 V buck-converter branches: motor and electronics
Current monitor	INA228, whole-vehicle placement
Battery temperature	10 kOhm NTC thermistor
Motors	2 TT gearmotors
Motor driver	TB6612FNG
Distance sensor	HC-SR04 on microservo
IMU	MPU9250 module, used primarily for accel/gyro/orientation functions
RFID	MFRC522
Displays	2x 0.96 in 128x64 OLED; MAX7219 8x8 matrix
Camera	Forward-facing Logitech USB webcam
Control UI	Flask browser dashboard
Vehicle OS	Raspberry Pi OS Lite
Pi language	Python
Vision	OpenCV Canny-edge-based left/right obstruction scoring
Pi control loop	20 ms / 50 Hz
Browser watchdog	0.50 s
Critical ECU timeout	0.50 s
OpenCV scoring interval	0.10 s nominal
Wall thresholds	40 cm slow, 20 cm stop positive command, 12 cm hard intervention
Current thresholds	4.75 A warn/reduce, 5.25 A stronger limit, 5.50 A shutdown
Overcurrent cooldown	1.5 s
Autonomous trigger	Approximately 32 cm
Autonomous reverse	Up to 25% for approximately 0.35 s
Autonomous turn	Up to 38% for approximately 0.55 s
Dimensions	24 x 19 x 19 cm
Mass	Approximately 872 g including battery
Runtime	Approximately 1 hour
Observed current	0.8-0.9 A idle; 1.2-1.3 A typical driving; approximately 1.6 A peak observed
Final control latency	Approximately 20-30 ms using direct IPv4 access
Estimated top speed	Approximately 4 ft/s, not encoder-verified
Approximate project cost	$300

Appendix B. CAN Message Reference

The following table reflects the final Raspberry Pi controller's CAN definitions. Multi-byte values are little-endian where noted. Rolling counters are included in several frames for command/telemetry sequencing.

ID	Byte Layout
0x100	B0-1: voltage mV (u16); B2-3: temperature 0.01 C (i16); B4-5: current mA (i16); B6-7 reserved/unused by Pi decoder.
0x200	B0-1: forward distance mm; B2-3: servo angle 0.1 deg; B4: flags including unlock; B5: sensor status flags; B6-7: rolling counter.
0x201	B0-1: ax in milli-g; B2-3: ay; B4-5: az; B6-7: rolling counter.
0x202	B0-1: gx in 0.01 deg/s; B2-3: gy; B4-5: gz; B6-7: rolling counter.
0x203	B0-1: roll in 0.01 deg; B2-3: pitch; B4-5: yaw; B6-7: rolling counter.
0x204	B0-1: left range mm; B2-3: center range; B4-5: right range; B6-7: rolling counter.
0x300	B0: actual left output i8%; B1: actual right output i8%; B2: enabled; B3: motor fault flags; B4-5: rolling counter; B6-7 reserved.
0x400	B0: left command i8%; B1: right command i8%; B2: mode; B3: speed limit; B4: E-stop/unlock flags; B5: safety flags; B6-7: command counter.
0x401	B0: desired lock state; B1: source (browser=1); B2-5 reserved; B6-7: command counter.

Appendix C. Development Tools & Authorship

Development Tools

The project used a mix of embedded, PCB, Linux, networking, and computer-vision tools. The goal was not only to make each subsystem work independently, but to learn how these tools fit together during full vehicle integration.

Tool / Platform	Use in Project
KiCad	Schematic capture, custom footprints, PCB layout, DRC, and manufacturing preparation.
JLCPCB	Fabrication of the three custom two-layer ECU PCB designs.
ESP32 / Arduino ecosystem	Embedded firmware, peripheral interfaces, CAN node logic, and local ECU behavior.
Raspberry Pi OS Lite	Headless vehicle-computer operating system.
Python	Supervisory controller, CAN processing, safety logic, drive control, and system state.
Flask	Local browser dashboard and JSON control/telemetry API.
SocketCAN	Linux-native CAN interface used by the Raspberry Pi.
OpenCV	USB camera capture and left/right visual obstruction analysis for autonomous mode.
Linux / SSH tools	Remote development and debugging using SSH, candump, ip link, nano, and device/interface inspection.

Authorship and Development Approach

I designed the system architecture, defined the ECU responsibilities and CAN protocol, wired and integrated the electronics, created all three PCB schematics and layouts, hand soldered and assembled the final boards, performed the mechanical assembly, and completed the final system integration and testing. The project uses standard open-source libraries for common hardware and software functions. Software development was iterative and included external review and troubleshooting assistance, while the hardware architecture, PCB designs, protocol structure, integration decisions, and final validation were completed by me.